

Práctica 1 · Mi primera app Android

Julio Carmona Diaz PMDM

07/10/2026



Índice

Título 1	3
Título 2	3
Título 3	3
Título 4	3



Introduccion:

ExplorarRutas es una aplicación dedicada a las personas que les gusta explorar el mundo y hacer distintas rutas y caminos al aire libre.

PARTE A:

Ventajas e inconvenientes

Limitación: Batería:

La app usará gps y al aire libre siendo de día y teniendo que mirarlo constantemente consumirá también mucho por el brillo, debemos evitar que haya aplicaciones en 2 plano.

Limitación: Memoria:

No podemos cargar cientos de fotos de rutas de golpe, habrá que usar imágenes optimizadas.

Limitación: conectividad:

Un senderista puede estar en una zona sin cobertura, la app debería poder mostrar al menos los datos básicos aunque no tenga cobertura.

Limitación: tamaño de pantalla:

Debe verse bien en un móvil o en una tablet.

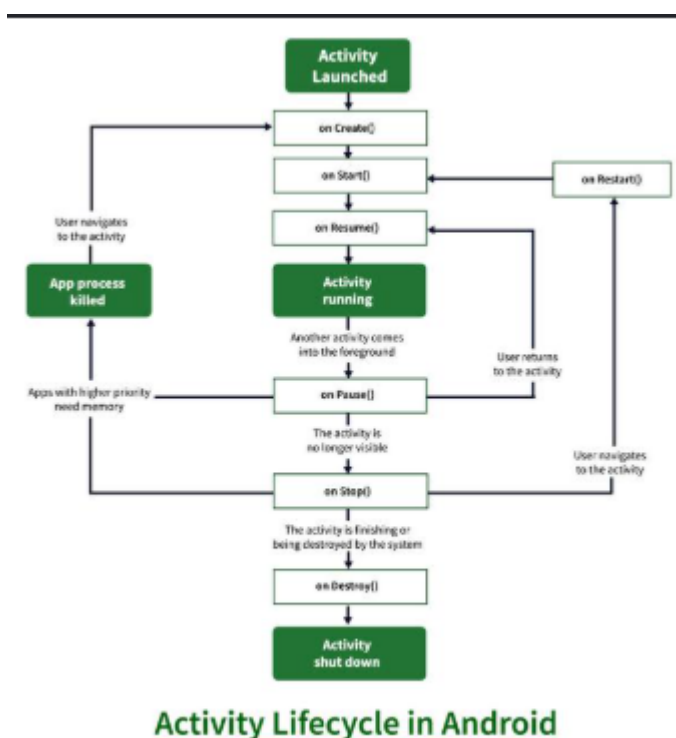
Limitación: Fragmentación:

Hay miles de dispositivos Android con diferentes versiones, lo configuraremos con API 24+ para cubrir la mayoría sin complicarnos.

Tabla comparativa:

Característica	Nativo	Híbrido	Multiplataforma
Lenguaje	Java/Kotlin	HTML,CSS,JS	Dart
Rendimiento	Alto	Medio/bajo	Alto
Acceso Hardware	Total	Limitado	Alto
Coste	Mayor	Menor	Medio
Ejemplo	Android studio	Ionic+cordova	Flutter

Ciclo de vida de una activity





PARTE B

Idea y público objetivo:

ExploraRutas es una aplicación hecha para gente que quiera explorar el mundo, sobre todo la naturaleza, el objetivo de la aplicación es centralizar información básica de cada ruta para que el usuario pueda planificar su excursión sin necesidad de consultar múltiples fuentes.

Objetivos medibles

Objetivo 1: lograr que el usuario encuentre una ruta de su interés lo más rápido posible .

Objetivo 2: Conseguir que el tiempo de carga sea inferior a 1.5 segundos

Objetivo 3: Alcanzar una tasa de retención del 20% en la primera semana, mediante una interfaz que permita guardar rutas favoritas.

Requisitos Funcionales (RF):

RF-01: la app debe mostrar una lista de rutas disponibles con su nombre, dificultad y distancia.

RF-02: Al pulsar sobre una ruta, se debe abrir una pantalla de detalle con su descripción completa.

RF-03: La app debe permitir filtrar las rutas por nivel de dificultad.

RF-04: El usuario debe de poder marcar una ruta como favorita.

RF-05: la app debe mostrar una imagen representativa de cada ruta.

RF-06: La app debe permitir compartir la ruta a través de otras aplicaciones.



Requisitos no funcionales (RNF):

RNF-01 (usabilidad): La interfaz debe ser legible al aire libre, usando un contraste de color adecuado y un tamaño de fuente mínimo de 14 sp.

RNF-02: (compatibilidad): la app debe de ser compatible con android 7.0 (API 24) o superior, cubriendo así más del 95% de los dispositivos activos.

RNF-03 (rendimiento): La pantalla principal debe cargar completamente en menos de 2 segundos en un dispositivo de gama media.

RNF-04 (Mantenibilidad): todo el código debe estar debidamente comentado y los recursos organizados en carpetas según su tipo.

Casos de uso:

caso 1: ver detalle de ruta

Actor: Usuario

Precondición : el usuario ha abierto la app y ve la lista de rutas.

Flujo Principal: el usuario pulsa sobre una ruta, luego el sistema carga la pantalla de detalle y luego el sistema muestra la imagen, descripción, dificultad y distancia.

Flujo Alternativo: si la ruta no tiene imagen, el sistema muestra una imagen vectorial por defecto.

Postcondición: el usuario visualiza la información completa de la ruta.



Caso 2: Filtrar rutas por dificultad.

Actor: Usuario

Precondición: El usuario está en la pantalla principal.

Flujo Principal: El usuario pulsa el botón de filtro luego selecciona "fácil" y por último el sistema actualiza la lista mostrando solo las rutas fáciles.

Flujo alternativo: Si no hay rutas fáciles, el sistema muestra un mensaje indicando "no hay rutas disponibles"

Postcondición: la lista se ha filtrado correctamente.



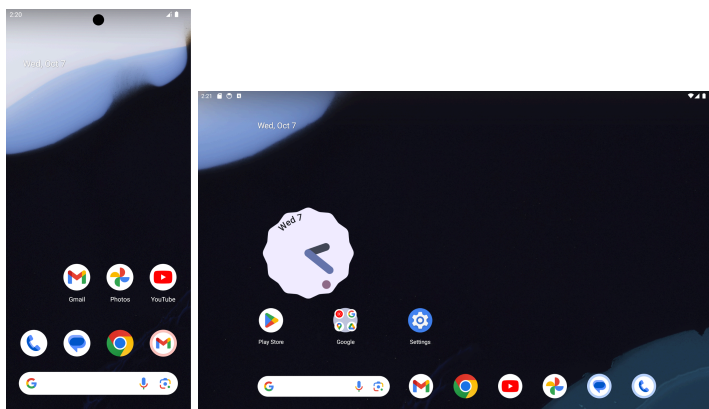
PARTE C:

Configuración del proyecto

El proyecto se ha creado en Android Studio con el nombre PMDM_P1_JulioCarmona y el paquete es.medac.julioarmona.app. Se ha utilizado la plantilla Empty Views Activity, lenguaje Java y una API mínima 24 (Android 7.0).

Dispositivos virtuales (AVD)

Dispositivo	Modelo	Resolucion	Densidad	Version API
Movil	Pixel6	1080*2400	420dpi	34
Tablet	pixel tablet	2560*1600	320dpi	34





Estructura del proyecto

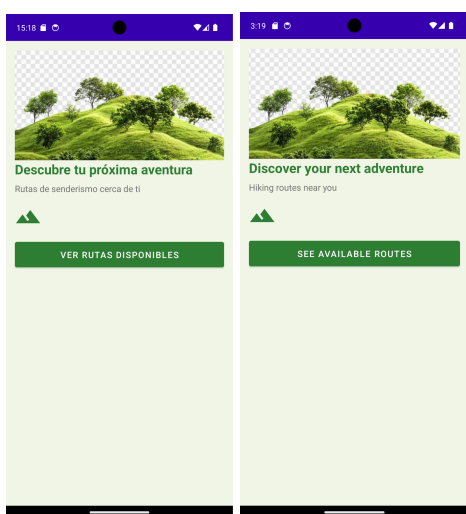
- AndroidManifest.xml: Archivo de configuración principal. Declara el nombre del paquete, los permisos, las Activities y el icono de la app.
- java/: Contiene el código fuente en Java, incluyendo MainActivity.java, que controla la lógica de la pantalla.
- res/: Carpeta de recursos dividida en:
 1. layout/: Archivos XML con el diseño visual de las pantallas.
 2. values/: Textos (strings.xml), colores (colors.xml), estilos (styles.xml) y temas (themes.xml).
 3. drawable/: Imágenes vectoriales y de mapa de bits.
 4. mipmap/: Iconos de la app en diferentes resoluciones.
- Gradle: Archivos build.gradle.kts que indican las dependencias y la versión del SDK.
- relación Activity-Layout: La Activity (Java) actúa como el "cerebro" y el layout (XML) es la "cara". En onCreate() se usa setContentView(R.layout.activity_main) para unir ambos.



PARTE D:

Textos e Internacionalización

Todos los textos de la app están centralizados en `res/values/strings.xml` (mínimo 8 strings), sin ningún texto escrito a mano en los layouts. Se ha utilizado un string-array para las opciones de filtro y un texto con parámetro (`%1$s`) para la duración de la ruta. La app está traducida al inglés mediante la carpeta `values-en/strings.xml`.



- **Vectorial (ic_montana.xml):** Creada con Vector Asset. Se usa para iconos y formas simples. Ocupa muy poco espacio, no pierde calidad al escalar y se adapta a cualquier densidad de pantalla.
- **Mapa de bits (foto_ruta.jpg):** Usada para la fotografía real de la ruta. Aunque ocupa más espacio, es necesaria para mostrar detalles fotográficos que un vector no puede capturar. Se ha optimizado para no aumentar el tamaño de la app.



Colores

- La paleta en colors.xml utiliza nombres semánticos (por función):
- color_primario (#2E7D32): Verde bosque, representa la naturaleza.
- color_primario_oscuro (#1B5E20): Para variaciones y estados pulsados.
- color_secundario (#8D6E63): Marrón tierra, evoca los caminos.
- color_fondo (#F1F8E9): Verde muy claro, facilita la lectura al aire libre.
- color_texto_principal (#212121): Gris casi negro para máximo contraste.
- color_texto_secundario (#757575): Gris medio para subtítulos.
- color_blanco (FFFFFF): Para textos sobre fondos oscuros.

Estilos y Tema

Se han creado dos estilos reutilizables en styles.xml:

- EstiloTitulo: Para los títulos de las pantallas (tamaño 24sp, negrita, color primario).
- EstiloBoton: Para los botones (fondo verde primario, texto blanco, padding de 12dp).

El tema de la app (Theme.ExploraRutas) hereda de Material Components y aplica nuestros colores personalizados. Se usan unidades dp para medidas y sp para textos.

Icono

Se ha creado un icono adaptativo propio con Image Asset (una montaña sobre fondo verde) y se ha aplicado en el AndroidManifest.xml. Los archivos generados se encuentran en las carpetas mipmap.



CONCLUSION PERSONAL:

Durante la realización de esta práctica he aprendido a estructurar un proyecto Android desde cero, diferenciando claramente entre la lógica (Activity) y la interfaz (Layout XML). La mayor dificultad ha sido comprender el ciclo de vida de una Activity y la gestión de recursos, especialmente la internacionalización con values-en. También me enfrenté a errores de compilación al principio por usar una plantilla incorrecta, pero gracias a la organización de recursos en carpetas y al uso de estilos, comprobé lo importante que es separar el contenido de la presentación. Me llevo como aprendizaje la importancia de no escribir textos a mano en los layouts y de usar nombres semánticos para los colores.

WEBGRAFIA: